

Unit 1

JAVASCRIPT



AI Project - SanviPop

Client Side Web Development
2º DAM/DAW
IES San Vicente 2026/2027

Index

Introduction.....	3
Login (login.html).....	3
Register (register.html).....	3
Products (index.html).....	3
Add product (add-product.html).....	4
Product detail (product-detail.html).....	4
User profile (profile.html).....	4
What to submit and grading.....	5

Introduction

You have been hired by a company to work as a front-end developer in a vanilla TypeScript project. You have asked for training programs and learning courses before starting but your boss just told you “Use the AI”.

You need to have NodeJS (install the LTS version) installed before you can do anything.

Front-end (HTML mockups)

You have been given a client project that you need to complete (**sanvipop-client.zip**)

Web services (back-end)

The back-end is also provided (**sanvipop-services.zip**). In the **README.md** file there's a description of every implemented endpoint (this information should be very useful for your AI agent).

To execute the back-end just execute **npm i** and **npm start** inside this project's main directory. Services should be running in <http://localhost:3000>. You can access <http://localhost:3000/reference> to see the services generated documentation using Swagger and Scalar API and try them if you want.

Implement the necessary TypeScript code to make the different sections work. Don't change the HTML structure unless you are told to do so. Use only AI tools like agents (Antigravity, OpenCode, etc.). There are plugins for many of these environments to use them in Visual Studio Code for example.

Login (login.html)

Make this login form work. Show feedback to the user if anything goes wrong, and redirect to index.html if everything is correct. If you are already logged in when you visit this page, redirect to index.html. Send user geolocation if available.

Register (register.html)

Make this register form work. Apart from the included validations, validate that both passwords are equal and the avatar file is an image and size no higher than 1MB. Send user geolocation.

Show feedback to the user if anything goes wrong, and redirect to login.html if everything is correct. If you are already logged in when you visit this page, redirect to index.html.

Products (index.html)

Get the products from the server and implement the filters. Products are paginated. Show only the button to load more when necessary. Products have links to navigate to the details page or the profile page of the seller.

Add product (add-product.html)

Make this form work. Apart from the included validations, validate that the image file is an image and size no higher than 2MB.

If there's a query param with an id (example: **add-detail.html?id=3**), this form should be a form to edit the product instead, showing the current product's info in the fields and the product's image (image input field won't be mandatory in this case).

Product detail (product-detail.html)

In this page we'll show a product's details. We should get the id from the URL (query param → example: **product-detail.html?id=3**). If there's no id or no product with this id, redirect to index.html.

- Show the product's information in this page.
- You should be able to bookmark this product (toggle the 'active' CSS class in the button)
- If not sold, you should be able to buy this product (don't implement any payment system, just call the service).
- If sold, show the reviews if available. Only if this transaction is not reviewed by you yet, you should be able to review the transaction.
- Show the map with the user location. We'll use Openlayers in combination with OpenStreet maps to show the map as it doesn't required any credit card info in order to use it (install necessary dependencies).
- Implement the function to delete a product if you are the seller and it's not been sold yet.

Display or hide the necessary info depending on the product's and transaction's status.

User profile (profile.html)

This page shows the info of a user. Optionally it could receive a query param with and id (user to show instead of the logged user).

You can only edit avatar, profile and password if the profile is yours (logged user). Forms are hidden (toggle that visibility using the corresponding buttons).

You'll see links to see different products related to this user (selling, sold, bought and bookmarked products). Implement this functionality in the index.html page.

When filtering products by a user, hide the filters form (won't work). Also show in the page what the user is seeing (example: Products sold by Pepito Martinez).

What to submit and grading

- Upload the completed project in a zip file, deleting the **node_modules** folder first.
- Submit a document in TXT or PDF format containing a brief report outlining which AI environment you used (editor, agent, etc.), the prompts (and files) you gave to the AI along with what you achieved at each step, and a final reflection on the process (whether it was complex, whether you met your goals, etc.).
- The grade for this project will be based primarily on the report (not on the code, as it is generated by the AI), and it will account for 10% of the unit project grade that we will complete later. Additionally, we will need this document to carry out a comparative analysis in the final project of the unit.
- Don't worry if you couldn't get it to work properly, as our future objective will be to compare this process of working with AI blindly versus working with a clear understanding of what we are doing later on."